Q-Learning on Gymnasium Taxi

Terrell Fleming
CPSC 4420 - Assignment 2
GitHub: https://github.com/terrellf17/taxi-qlearning

A. Environment

I picked the Taxi environment (Taxi-v4 in Gymnasium). It's a 5x5 grid with some walls in it. A passenger is waiting at one of four colored spots (R, G, Y, B), and the taxi has to drive over, pick them up, then drive to their destination and drop them off.

The state space is discrete with 500 states: 25 taxi positions x 5 passenger locations (the 4 spots plus inside the taxi) x 4 destinations. Since it's already discrete I didn't need to discretize anything, the Q-table is just 500 rows. There are 6 actions: move south, north, east, west, pick up, and drop off.

The transitions are deterministic, so the same action in the same state always does the same thing. If the taxi drives into a wall it just stays where it is. For rewards, every step is -1, a correct drop-off is +20, and trying to pick up or drop off at the wrong place is -10. The episode ends (terminated) when the passenger is dropped off, and it gets cut off (truncated) after 200 steps. I used the default settings and didn't use the action mask the environment gives you.

B. Q-Learning Implementation

The agent is written from scratch with numpy, no RL libraries (qlearn/agent.py). The Q-table starts at all zeros. To pick actions it uses epsilon-greedy: with probability ε it takes a random action, otherwise it takes the action with the highest Q-value (ties are broken randomly). After each step it updates the table with:

```
Q(s,a) <- Q(s,a) + α [ r + γ max Q(s',a') - Q(s,a) ]
```

If the episode terminated (the passenger got delivered), there's no next state, so the target is just r. If the episode only hit the 200 step limit, that's not a real end state, so it still uses γ max Q(s',a'). After every episode epsilon decays: ε = max(ε_min, ε_start x decay^episode).

All the hyperparameters (α, γ, ε start, ε min, ε decay, number of episodes) are in one config class and can be changed from the command line, like python -m qlearn.train --alpha 0.5. For the experiments I trained every setting with 10 different random seeds and averaged them. Every 50 episodes I also tested the greedy policy (ε = 0) so I could see how good the actual learned policy was, not just the training reward.

C. Baseline

Baseline settings: α = 0.1, γ = 0.99, ε starts at 1.0 and decays by 0.999 each episode down to 0.05 (it hits 0.05 around episode 3,000), 5,000 episodes.

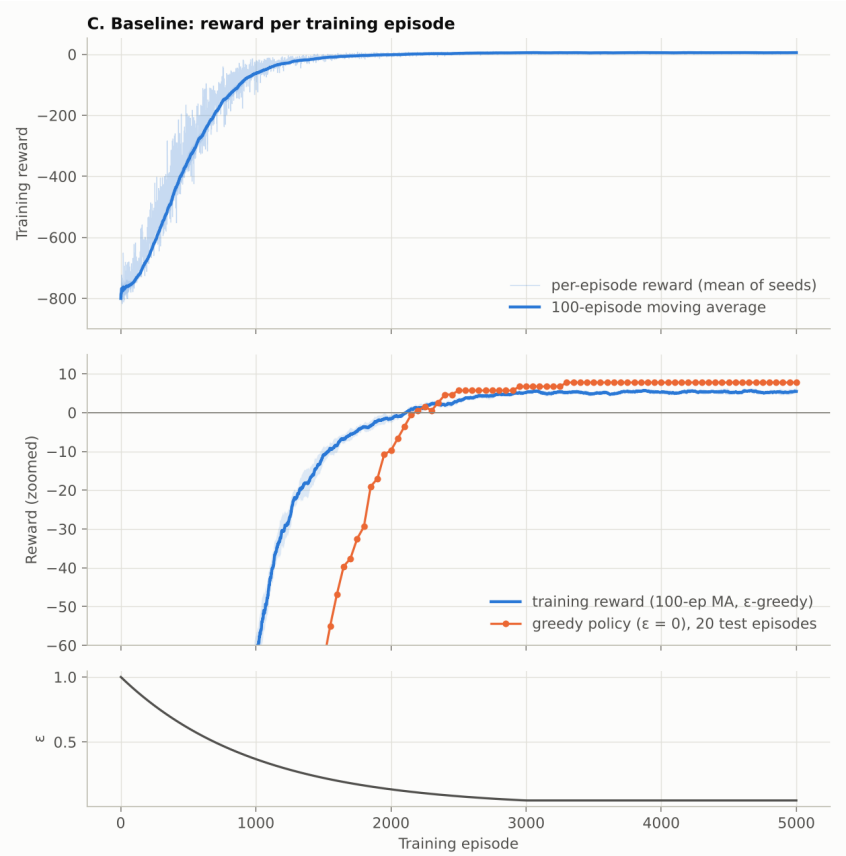


Figure 1. Baseline reward per episode (top), zoomed in with the greedy policy (middle), and epsilon (bottom).

At the start the agent is basically random. The first 100 episodes averaged about -745 reward and almost all of them hit the 200 step limit. The average reward goes above 0 around episode 2,050, and the greedy policy becomes optimal around episode 2,370. At the end the greedy policy got 8.05 on the test episodes, which is the same as the optimal policy, with 100% success.

The training reward levels off around 5.3 instead of 8 because ε is still 0.05, so 1 out of 20 moves is random. That's why I also tracked the greedy policy separately. The greedy line lags behind at first because the +20 reward has to work its way back through a ~13 step route, and with $\alpha = 0.1$ each update only moves the Q-value 10% of the way.

D. Learning Rate

I tried $\alpha = 0.01$, 0.1, 0.5, and 0.9 and kept everything else the same as the baseline.

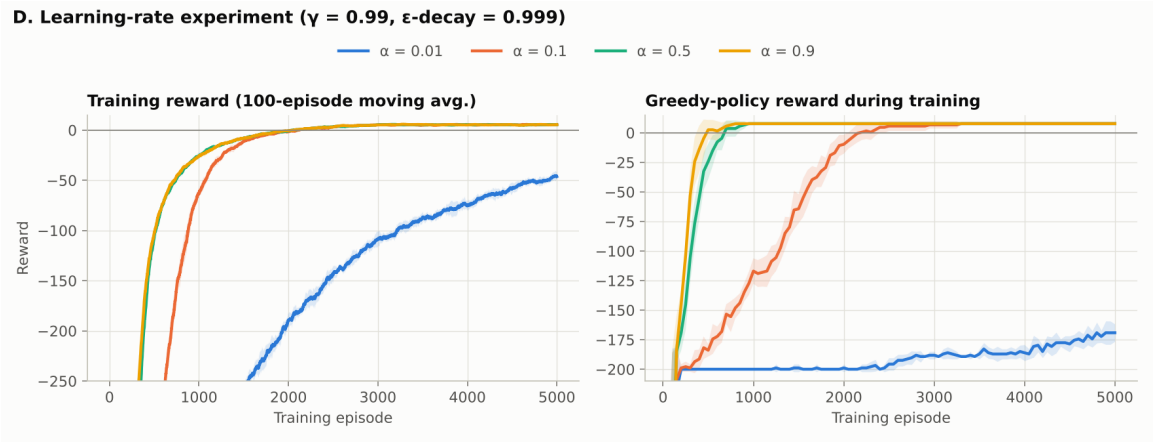


Figure 2. Different learning rates. Left: training reward. Right: greedy policy reward.

α	Episodes until optimal	Greedy reward	Success rate
0.01	never	-176.8	11%
0.1	~2,370	8.05	100%
0.5	~745	8.01	100%
0.9	~515	8.02	100%

A bigger learning rate learned way faster. α = 0.9 was about 4.6 times faster than 0.1. α = 0.01 was way too slow and was still failing most episodes after 5,000 episodes. Every α from 0.1 up ended at the optimal reward and stayed stable. A high α works fine here because Taxi is deterministic, so there's no randomness in the rewards that needs to be averaged out. That changes once noise is added in part H.

E. Discount Factor

I tried γ = 0.5, 0.7, 0.9, and 0.99 with α = 0.1.

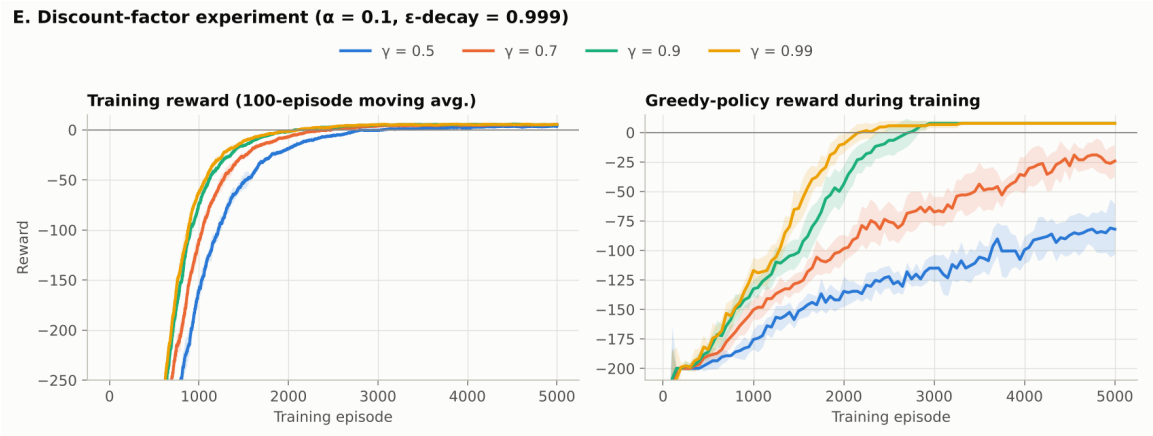


Figure 3. Different discount factors.

γ	Episodes until optimal	Greedy reward	Success rate
0.5	never	-64.8	65%
0.7	only 1 of 10 seeds	-20.1	86%
0.9	~2,700	8.05	100%
0.99	~2,370	8.05	100%

γ = 0.9 and 0.99 both worked. γ = 0.5 and 0.7 didn't. γ basically controls how far ahead the agent cares about. With γ = 0.5, the +20 reward 10 steps away is worth 20 x 0.5^10, which is about 0.02, so the agent can't really see it. Since a route takes around 13 steps, the agent can't tell the difference between good moves and bumping into walls and ends up going in circles. It still learned short-term stuff like not doing illegal pickups, which is why success wasn't 0. A γ close to 1 values the future reward almost as much as an immediate one, which is what you need for this task.

F. Epsilon Decay

For the extra hyperparameter I changed how fast epsilon decays: 0.99, 0.995, 0.999, and 0.9995 per episode.

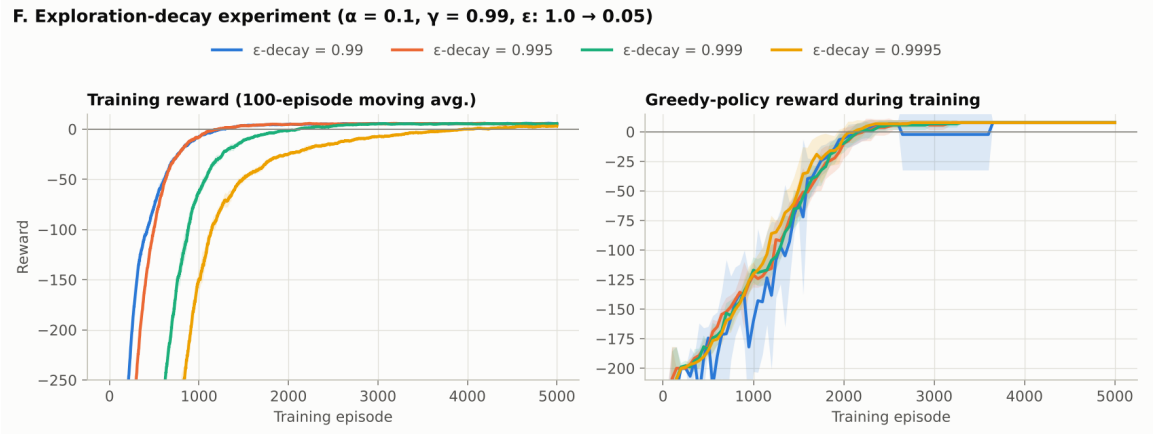


Figure 4. Different epsilon decay rates (α = 0.1, γ = 0.99).

Faster decay gave a better training reward (5.55 vs 2.66 over the last 500 episodes) since the agent stops making random moves sooner. But it didn't really make the greedy policy learn faster, they all got there around 2,200 to 2,400 episodes. With the fastest decay (0.99), one seed's policy got worse for about 1,000 episodes, which is the dip in the right graph, because some states weren't being explored enough to fix their values. The slowest decay ended with the most accurate Q-table. So exploring longer costs some reward during training but gives a more reliable policy.

G. Best Agent

To pick the best setup I used γ = 0.99 (the best from part E) and then tried every combination of α = 0.1, 0.5, 0.9 and ε decay = 0.99, 0.995, 0.999, 0.9995, with 10 seeds each. I picked the one with the highest greedy reward on 100 validation episodes, and if they tied I picked the one that learned fastest. The best was **α = 0.9, γ = 0.99, ε decay = 0.9995**. It became optimal around episode 435.

Then I tested it with exploration turned off (ε = 0) on 100 different test episodes for each of the 10 agents, so 1,000 episodes total.

	Mean reward	Success rate	Avg steps	Illegal moves	Episodes to learn
Baseline	7.578	100%	13.4	0	~2,370
Best	7.58	100%	13.4	0	~435
Optimal (value iteration)	7.58	100%	13.4	0	-

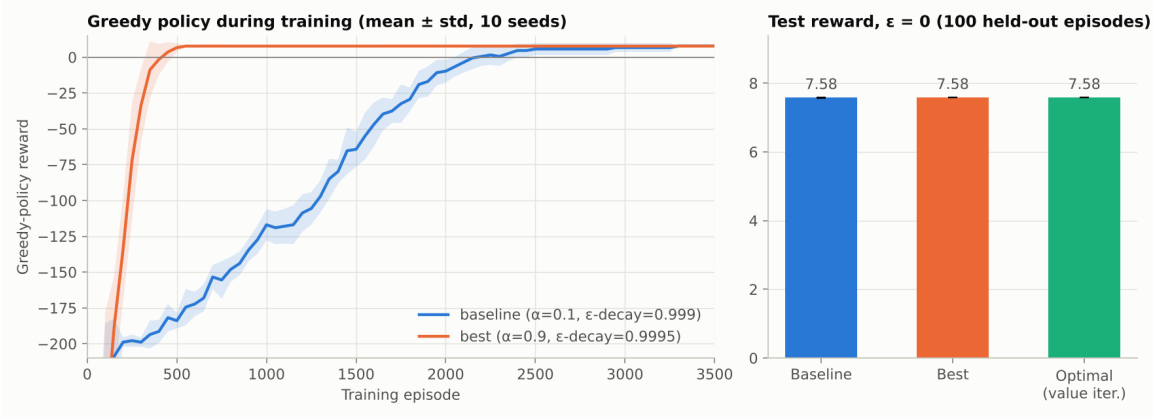


Figure 5. Greedy reward during training (left) and final test reward (right).

Both the baseline and the best agent ended up at the optimal policy. The difference is how fast they got there: about 435 episodes vs 2,370, so roughly 5x faster. One downside is the best setup explores longer, so its reward during training was lower (2.81 vs 5.34).

H. Stochastic Environment

Taxi is deterministic, so I added my own noise. With probability p, the agent's chosen action gets swapped for a random different action. I tested p = 0, 0.10, 0.25, and 0.40.

First I tried the best setup from part G (α = 0.9) and it fell apart under noise, it got -94 at p = 0.10. With noise the rewards are random, and a high learning rate just keeps chasing the latest result instead of averaging. α = 0.1 worked much better, so I used α = 0.1 for the rest of this part.

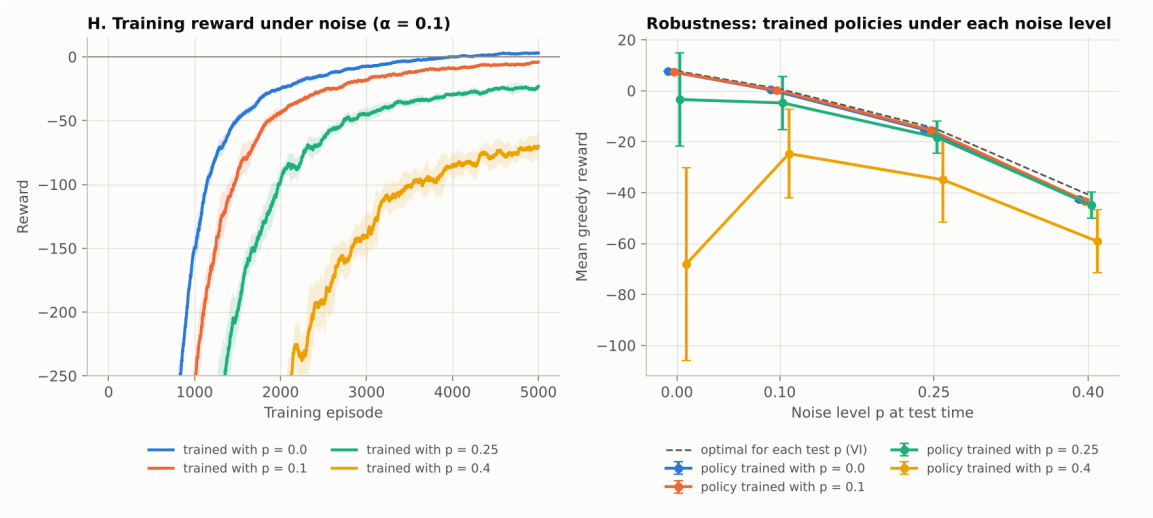


Figure 6. Left: training with noise. Right: each trained policy tested at every noise level. Dashed line is the best possible.

Trained at ↓ / Tested at →	**p = 0**	**p = 0.10**	**p = 0.25**	**p = 0.40**
p = 0 (original)	7.74	0.34	-15.31	-42.70
p = 0.10	7.29	0.24	-15.47	-43.26
p = 0.25	-3.46	-4.79	-18.31	-44.99
p = 0.40	-68.11	-24.70	-34.98	-59.14
Best possible	7.74	0.70	-14.62	-40.77

More noise made learning slower. It took about 2,185, 2,645, and 3,715 episodes to learn at p = 0, 0.10, and 0.25, and at p = 0.40 only 2 out of 10 seeds got there. The final reward also dropped a lot, but most of that can't be avoided, since even the best possible policy only gets -40.8 at p = 0.40 because the random moves add extra steps and penalties.

The original policy trained with no noise actually held up the best. At every noise level it was within 2 points of the best possible score, and it did better than the policies trained with noise. That's because this kind of noise doesn't change which action is best, it just makes the learning harder. The policies trained at p = 0.40 ended up depending on the noise: with no noise they only got -68 and failed 19% of the time, because they would keep bumping into the same wall and only the noise would get them unstuck.

Conclusions

Q-learning was able to solve Taxi completely, and the best agent matched the optimal policy on every test episode. The learning rate controls how fast it learns, and a high one is best when there's no noise but needs to be small when there is. The discount factor has to be close to 1 or the agent can't see the reward at the end of the route. Epsilon decay trades reward during training for a more accurate and stable policy at the end.